

MÉTODOS MODERNOS DE GRADIENT BOOSTING

Adriana Haydeé Contreras Peruyero

*Escuela Nacional de Estudios Superiores Unidad Juriquilla,
UNAM*
haydeeperuyero@unam.com

Temas Selectos de Aprendizaje
de Máquina
Semestre 2027-1



1 Repaso

2 Boosting

3 Métodos modernos de Gradient Boosting

REPASO

Recordatorio: árboles de decisión

Un **árbol de decisión** divide recursivamente el espacio de predictores mediante reglas sencillas.

$$x_j < c \quad \Longrightarrow \quad \begin{cases} \text{región izquierda} \\ \text{región derecha} \end{cases}$$

- ▶ Cada nodo interno representa una regla de división.
- ▶ Las hojas contienen la predicción final.
- ▶ Pueden utilizarse tanto para **regresión** como para **clasificación**.

Ejemplo



Árboles de regresión y clasificación

Regresión

La respuesta es cuantitativa:

$$Y \in \mathbb{R}.$$

En cada hoja se produce un valor numérico, por ejemplo, el promedio de las observaciones de entrenamiento que caen en ella.

Clasificación

La respuesta es categórica:

$$Y \in \{C_1, \dots, C_K\}.$$

En cada hoja podemos asignar una clase o estimar probabilidades de pertenencia.

Pero hay un problema...

Un solo árbol puede ser muy sensible a los datos utilizados para construirlo.

Una limitación de los árboles

Los árboles de decisión pueden presentar **alta varianza**.

Si dividimos aleatoriamente los datos de entrenamiento en dos conjuntos y construimos un árbol en cada uno, podemos obtener árboles bastante diferentes.

$$\mathcal{D}_1 \longrightarrow \hat{f}_1 \qquad \mathcal{D}_2 \longrightarrow \hat{f}_2$$

aunque $\mathcal{D}_1, \mathcal{D}_2 \sim$ misma población.

Idea

Pequeños cambios en los datos pueden producir cambios importantes en la estructura del árbol y, por tanto, en sus predicciones.

Si un árbol tiene mucha varianza, ¿qué podríamos hacer para estabilizarlo?

De un árbol a un ensamble

Una idea clásica para reducir varianza consiste en **promediar varias estimaciones**.

Si tenemos B predictores

$$\hat{f}^1(x), \hat{f}^2(x), \dots, \hat{f}^B(x),$$

podemos construir

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^b(x).$$

Intuición

Si los modelos individuales cometen errores diferentes, al combinarlos podemos obtener una predicción más estable.

¿De dónde obtenemos muchos conjuntos de entrenamiento?

Idealmente quisiéramos tener:

$$\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \dots, \mathcal{D}^{(B)}$$

y entrenar un árbol diferente con cada conjunto.

¿De dónde obtenemos muchos conjuntos de entrenamiento?

Idealmente quisiéramos tener:

$$\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \dots, \mathcal{D}^{(B)}$$

y entrenar un árbol diferente con cada conjunto.

Pero normalmente sólo tenemos un conjunto de datos.

¿De dónde obtenemos muchos conjuntos de entrenamiento?

Idealmente quisiéramos tener:

$$\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \dots, \mathcal{D}^{(B)}$$

y entrenar un árbol diferente con cada conjunto.

Pero normalmente sólo tenemos un conjunto de datos.

La solución es utilizar **bootstrap**:

$$\mathcal{D} \longrightarrow \mathcal{D}^{*(1)}, \mathcal{D}^{*(2)}, \dots, \mathcal{D}^{*(B)}.$$

Cada $\mathcal{D}^{*(b)}$ se obtiene muestreando observaciones de $\mathcal{D}$ **con reemplazo**.

Bagging: Bootstrap Aggregating

Bagging combina dos ideas:

Bootstrap

+

Aggregating

1. Generamos B muestras bootstrap.

$$\mathcal{D}^{*(1)}, \dots, \mathcal{D}^{*(B)}.$$

2. Entrenamos un árbol en cada muestra:

$$\hat{f}^{*1}, \dots, \hat{f}^{*B}.$$

3. Combinamos sus predicciones.

Bagging

Obtenemos así:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

Bagging en regresión y clasificación

La forma de combinar los árboles depende del problema.

Regresión

Promediamos:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x).$$

Clasificación

Cada árbol produce una clase:

$$\hat{C}_1(x), \dots, \hat{C}_B(x).$$

Una posibilidad es utilizar **voto mayoritario**.

$$\hat{C}(x) = \text{mode}\{\hat{C}_1(x), \dots, \hat{C}_B(x)\}.$$

De Bagging a Random Forest

Bagging introduce variación mediante diferentes muestras bootstrap.

Pero supongamos que existe un predictor **muy fuerte**.

De Bagging a Random Forest

Bagging introduce variación mediante diferentes muestras bootstrap.

Pero supongamos que existe un predictor **muy fuerte**.

Muchos árboles pueden elegir ese mismo predictor en sus primeras divisiones:

$$\text{árbol 1} \approx \text{árbol 2} \approx \dots \approx \text{árbol B.}$$

Entonces sus predicciones estarán **correlacionadas**.

Problema

Promediar modelos muy correlacionados reduce menos la varianza que promediar modelos que cometen errores más diversos.

Random Forest

Random Forest introduce una fuente adicional de aleatoriedad.

En cada división del árbol:

1. se selecciona aleatoriamente un subconjunto de predictores;
2. solamente esos predictores pueden competir para realizar el split.

Si tenemos p predictores, consideramos únicamente

$$m < p$$

en cada división.

Objetivo

Decorrelacionar los árboles para que su combinación reduzca más eficazmente la varianza.

BOOSTING

Una filosofía diferente: Boosting

En **bagging**, los árboles se construyen aproximadamente de manera independiente:

$$\begin{array}{ccc} \mathcal{D}^{*(1)} & \longrightarrow & f_1 \\ \mathcal{D}^{*(2)} & \longrightarrow & f_2 \\ \vdots & & \vdots \\ \mathcal{D}^{*(B)} & \longrightarrow & f_B \end{array} \quad \longrightarrow \quad \text{combinar.}$$

Una filosofía diferente: Boosting

En **bagging**, los árboles se construyen aproximadamente de manera independiente:

$$\begin{array}{ccc} \mathcal{D}^{*(1)} & \longrightarrow & f_1 \\ \mathcal{D}^{*(2)} & \longrightarrow & f_2 \\ \vdots & & \vdots \\ \mathcal{D}^{*(B)} & \longrightarrow & f_B \end{array} \quad \longrightarrow \quad \text{combinar.}$$

En **boosting**, los modelos se construyen **secuencialmente**:

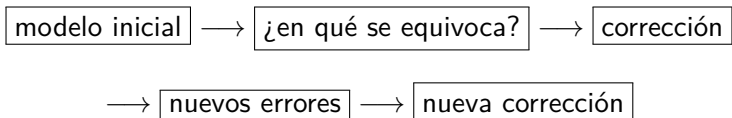
$$f_1 \longrightarrow f_2 \longrightarrow f_3 \longrightarrow \cdots \longrightarrow f_B.$$

Diferencia fundamental

Cada nuevo modelo utiliza información de los modelos construidos anteriormente.

Boosting: aprender de los errores anteriores

En lugar de construir un único árbol grande y complejo, boosting construye una secuencia de árboles relativamente sencillos.



Cada nuevo árbol intenta mejorar aquello que el ensamble actual todavía **no modela adecuadamente**.

Idea clave

Los árboles ya no son independientes: el árbol siguiente depende de lo que aprendieron los anteriores.

Bagging vs. Boosting

	Bagging	Boosting
Construcción	Paralela/independiente	Secuencial
Datos	Muestras bootstrap	Depende del método
Relación entre modelos	Débil	Cada modelo depende de los anteriores
Idea principal	Promediar para reducir varianza	Corregir errores progresivamente
Ejemplo	Random Forest	Gradient Boosting

De Boosting a Gradient Boosting

La idea general es:

modelo $\longrightarrow$ error $\longrightarrow$ corrección.

Pero:

¿qué debe aprender exactamente el siguiente árbol?

Gradient Boosting proporciona una respuesta a esta pregunta utilizando una **función de pérdida** y su **gradiente**.

Gradient Boosting: construcción aditiva

Gradient Boosting construye el predictor de manera **secuencial y aditiva**.

Partimos de un predictor inicial $F_0(x)$ y añadimos correcciones:

$$F_1(x) = F_0(x) + h_1(x),$$

$$F_2(x) = F_1(x) + h_2(x),$$

y continuamos sucesivamente hasta obtener

$$F_M(x) = F_0(x) + \sum_{m=1}^M h_m(x)$$

Gradient Boosting: construcción aditiva

Los modelos h_m suelen ser **árboles de decisión pequeños**.

Pregunta fundamental

¿Qué debe aprender el siguiente árbol h_m ?

Comencemos con un problema de regresión

Consideremos un conjunto de entrenamiento

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n.$$

Supongamos que nuestro modelo actual es $F_{m-1}(x)$.

Para cada observación podemos calcular su **residuo**:

$$r_i^{(m-1)} = y_i - F_{m-1}(\mathbf{x}_i)$$

- ▶ $r_i > 0$: el modelo está **subestimando**.
- ▶ $r_i < 0$: el modelo está **sobreestimando**.
- ▶ $r_i \approx 0$: la predicción es cercana al valor observado.

Definición de residuo

El **residuo** de la observación i es la diferencia entre su valor observado y el valor predicho:

$$r_i = y_i - \hat{y}_i.$$

Un residuo positivo indica que el modelo está subestimando la respuesta, mientras que un residuo negativo indica que la está sobreestimando.

¿Qué aprende el siguiente árbol?

La idea es entrenar el siguiente árbol para aproximar los **errores del modelo actual**:

$$h_m(\mathbf{x}_i) \approx r_i^{(m-1)}.$$

Como $r_i^{(m-1)} = y_i - F_{m-1}(\mathbf{x}_i)$,

el árbol intenta aprender

$$h_m(\mathbf{x}_i) \approx y_i - F_{m-1}(\mathbf{x}_i).$$

Después actualizamos:

$$F_m(x) = F_{m-1}(x) + h_m(x)$$

¿Por qué funciona esta corrección?

Si el nuevo árbol aprende aproximadamente el residuo,

$$h_m(\mathbf{x}_i) \approx y_i - F_{m-1}(\mathbf{x}_i),$$

entonces

$$\begin{aligned} F_m(\mathbf{x}_i) &= F_{m-1}(\mathbf{x}_i) + h_m(\mathbf{x}_i) \\ &\approx F_{m-1}(\mathbf{x}_i) + y_i - F_{m-1}(\mathbf{x}_i) \\ &= y_i. \end{aligned}$$

Idea clave

Cada árbol intenta modelar aquello que el ensamble construido hasta ese momento todavía no ha logrado explicar.

Ejemplo: correcciones sucesivas

Supongamos que para una observación

$$y = 10.$$

El modelo actual predice

$$F_1(x) = 6.$$

Entonces el residuo es

$$r^{(1)} = 10 - 6 = 4.$$

Ejemplo: continuación

El siguiente árbol produce una corrección

$$h_2(x) = 3.$$

Por tanto,

$$F_2(x) = 6 + 3 = 9.$$

y ahora

$$r^{(2)} = 10 - 9 = 1.$$

Ejemplo: continuar corrigiendo

Nuestra predicción actual es

$$F_2(x) = 9$$

y todavía tenemos un residuo

$$r^{(2)} = 1.$$

Supongamos que el siguiente árbol propone

$$h_3(x) = 0.8.$$

Ejemplo: Continuación

Entonces

$$F_3(x) = 9 + 0.8 = 9.8$$

y

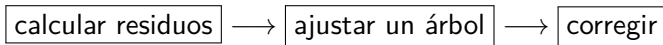
$$r^{(3)} = 10 - 9.8 = 0.2.$$

Lo podemos ver como:

$$6 \xrightarrow{+3} 9 \xrightarrow{+0.8} 9.8 \xrightarrow{\cdots} 10$$

¿Por qué se llama Gradient Boosting?

Hasta ahora parece que el procedimiento consiste simplemente en:



Pero esto plantea una pregunta:

¿qué ocurre si utilizamos una función de pérdida diferente?

La interpretación mediante residuos ordinarios corresponde a un caso particular.

Idea

Gradient Boosting generaliza esta estrategia utilizando el **gradiente de la función de pérdida**.

Recordatorio: descenso por gradiente

Supongamos que queremos minimizar una función

$$J(\boldsymbol{\theta}).$$

En descenso por gradiente actualizamos:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla J(\boldsymbol{\theta}_t)$$

donde $\eta > 0$ controla el tamaño del paso.

- ▶ ∇J apunta hacia una dirección de crecimiento.
- ▶ $-\nabla J$ proporciona una dirección de descenso.

posición actual $\xrightarrow{-\eta \nabla J}$ menor pérdida

Del descenso por gradiente al Gradient Boosting

En descenso por gradiente modificamos un vector de parámetros:

$$\theta \longrightarrow \theta - \eta \nabla J.$$

En Gradient Boosting estamos construyendo una **función**: $F(x)$.

En cada iteración buscamos una nueva función $h_m(x)$ que nos ayude a movernos en una dirección que **disminuya la pérdida**.

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

Diferencia conceptual

No estamos buscando directamente un nuevo vector de parámetros: estamos añadiendo una nueva función al modelo.

La función de pérdida

Sea

$$L(y, F(x))$$

una función que mide el error de una predicción.

Sobre el conjunto de entrenamiento queremos disminuir

$$\mathcal{L}(F) = \sum_{i=1}^n L(y_i, F(\mathbf{x}_i)).$$

En la iteración m ya tenemos un modelo

$$F_{m-1}(x).$$

La función de pérdida

La pregunta es:

¿en qué dirección debemos modificar sus predicciones para reducir la pérdida?

Para cada observación calculamos

$$r_{im} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}$$

Estos valores reciben el nombre de **pseudo-residuos**.

Representan una dirección local en la que conviene modificar las predicciones para disminuir la pérdida.

El siguiente árbol se ajusta para aproximarlos:

$$h_m(\mathbf{x}_i) \approx r_{im}.$$

Gradient Boosting: una iteración

En cada iteración:

1. Calculamos los pseudo-residuos:

$$r_{im} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}.$$

2. Entrenamos un árbol:

$$h_m(\mathbf{x}_i) \approx r_{im}.$$

3. Actualizamos:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x).$$

Interpretación

Cada árbol aproxima una dirección relacionada con el **gradiente negativo de la pérdida**.

Pérdida cuadrática

Consideremos la pérdida

$$L(y, F) = \frac{1}{2}(y - F)^2.$$

Derivamos respecto a F :

$$\begin{aligned}\frac{\partial L}{\partial F} &= \frac{1}{2}2(y - F)(-1) \\ &= -(y - F).\end{aligned}$$

Por tanto,

$$-\frac{\partial L}{\partial F} = y - F.$$

Los residuos aparecen naturalmente

Para pérdida cuadrática,

$$-\frac{\partial L}{\partial F} = y - F.$$

Por tanto, en la iteración m ,

$$r_{im} = y_i - F_{m-1}(\mathbf{x}_i).$$

Pero esto es precisamente el **residuo ordinario**.

Para pérdida cuadrática, pseudo-residuo = residuo ordinario.

Entrenar árboles sobre los residuos es un caso particular de seguir el gradiente negativo de una función de pérdida.

Learning rate

La actualización se escribe como

$$F_m(x) = F_{m-1}(x) + \eta h_m(x),$$

donde

$$0 < \eta \leq 1$$

es el **learning rate**.

Supongamos que el árbol propone una corrección

$$h_m(x) = 2.$$

Entonces:

$$\eta = 1 \implies \eta h_m(x) = 2,$$

mientras que

$$\eta = 0.1 \implies \eta h_m(x) = 0.2.$$

Shrinkage: aprender más lentamente

Reducir la contribución de cada nuevo árbol se denomina **shrinkage**.

$$F_m(x) = F_{m-1}(x) + \underbrace{\eta}_{\text{learning rate}} h_m(x).$$

Un learning rate pequeño produce correcciones más conservadoras.

$$\text{learning_rate} \downarrow \iff \text{n_estimators} \uparrow$$

Idea

Aprender más lentamente suele requerir construir más árboles.

Complejidad de los árboles

Cada árbol h_m **no necesita resolver todo el problema.**

Sólo necesita aprender una corrección útil.

Por ello, en Gradient Boosting suelen utilizarse árboles relativamente pequeños.

Algunos hiperparámetros importantes son:

- ▶ `max_depth`,
- ▶ `min_samples_split`,
- ▶ `min_samples_leaf`,
- ▶ `max_leaf_nodes`.

complejidad global $\longleftrightarrow$ árbol $+ \eta$ + número de árboles

Stochastic Gradient Boosting

Hasta ahora cada árbol ha utilizado todas las observaciones.

Podemos introducir aleatoriedad entrenando cada árbol con sólo una **fracción aleatoria** de los datos.

Si $0 < s < 1$, cada árbol utiliza aproximadamente sn observaciones.

En `GradientBoostingRegressor`:

$$\text{subsample} = 0.8$$

significa utilizar aproximadamente el 80% de las observaciones en cada iteración.

Muestreo como regularización

Introducir aleatoriedad mediante

$$\text{subsample} < 1$$

puede actuar como una forma adicional de **regularización**.

- ▶ Cada árbol observa sólo una parte de las observaciones.
- ▶ Se introduce diversidad entre los árboles.
- ▶ Puede reducirse la varianza del ensamble.
- ▶ Cada árbol puede resultar más barato de construir.

Importante

Esto no convierte Gradient Boosting en bagging.

Los árboles siguen construyéndose **secuencialmente** y cada nuevo árbol depende del ensamble anterior.

¿Más árboles siempre es mejor?

Al agregar árboles, el error de entrenamiento normalmente continúa disminuyendo.

$$E_{\text{train}}(1) > E_{\text{train}}(2) > E_{\text{train}}(3) > \dots$$

Pero el comportamiento sobre datos no observados puede ser diferente.

Podemos tener:

$$E_1 > E_2 > \dots > E_{m^*}$$

y posteriormente

$$E_{m^*+1} > E_{m^*}.$$

Entonces,

$$m^* = \arg \min_m E_m.$$

Importante

Más árboles no implica automáticamente un mejor modelo.

Early stopping

Una posibilidad es observar el desempeño de validación a medida que aumenta el número de árboles:

$$F_1, F_2, \dots, F_m, \dots$$

y calcular

$$E_1, E_2, \dots, E_m, \dots$$

De esta forma podemos detener cuando continuar entrenando deja de mejorar validación.

Esta estrategia se conoce como **early stopping**.

Interpretación

El número de árboles también puede verse como una cantidad que debemos seleccionar de acuerdo con el desempeño fuera del entrenamiento.

Gradient Boosting en Scikit-Learn

Para regresión podemos utilizar:

```
from sklearn.ensemble import GradientBoostingRegressor

modelo = GradientBoostingRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=2,
    random_state=42
)

modelo.fit(X_train, y_train)
```

Tres hiperparámetros fundamentales

Hiperparámetro	¿Qué controla?
n_estimators	Número de árboles
learning_rate	Contribución de cada nuevo árbol
max_depth	Complejidad de cada árbol

No actúan independientemente

Por ejemplo,

learning_rate ↓ ⇒ n_estimators ↑

puede ser necesario.

¿Qué tenemos hasta ahora?

Gradient Boosting construye

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

donde cada nuevo árbol:

1. depende del ensamble construido anteriormente;
2. aproxima una dirección relacionada con el gradiente negativo;
3. realiza una corrección controlada por el learning rate.

¿Qué tenemos hasta ahora?

Pero todavía aparecen varios retos:

- ▶ el entrenamiento es secuencial;
- ▶ pueden requerirse muchos árboles;
- ▶ varios hiperparámetros interactúan;
- ▶ debemos controlar el sobreajuste;
- ▶ con datos grandes el entrenamiento puede ser costoso.

¿Cómo podemos mejorar esta idea?

MÉTODOS MODERNOS DE GRADIENT BOOSTING

Métodos modernos de Gradient Boosting

La idea fundamental permanece:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x).$$

Los métodos modernos mantienen la construcción secuencial, pero incorporan mejoras para:

- ▶ controlar mejor la **complejidad** del modelo y mejorar la **eficiencia computacional**;
- ▶ incorporar mecanismos de **regularización**;
- ▶ optimizar la construcción de los árboles.

Uno de los métodos más importantes es:

XGBoost: Extreme Gradient Boosting

¿XGBoost es otro algoritmo completamente diferente?

No. XGBoost pertenece a la familia de Gradient Boosting.

Ambos construyen un modelo aditivo:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x).$$

La diferencia está en **cómo se optimiza y regulariza** la construcción de los nuevos árboles.

Idea

XGBoost puede entenderse como una implementación altamente optimizada y regularizada de Gradient Boosting con árboles.

XGBoost: función objetivo

En Gradient Boosting queremos reducir una función de pérdida.

XGBoost hace explícito además un término que penaliza la **complejidad del modelo**:

$$\mathcal{J} = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{m=1}^M \Omega(h_m)$$

Tenemos dos objetivos:

Ajuste

$$\sum_{i=1}^n L(y_i, \hat{y}_i)$$

Queremos predecir bien los datos.

Regularización

$$\sum_{m=1}^M \Omega(h_m)$$

Queremos controlar la complejidad.

Ajuste vs. complejidad

La estructura pérdida + penalización no es nueva.

Por ejemplo, en regresión regularizada:

$$\underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{ajuste}} + \lambda \underbrace{\sum_{j=1}^p \beta_j^2}_{\text{complejidad}} .$$

En XGBoost la misma filosofía se aplica a los **árboles**.

Objetivo

No buscamos solamente disminuir el error de entrenamiento:
también queremos evitar árboles innecesariamente complejos.

Regularización de los árboles

Una forma/ejemplo de penalizar la complejidad de un árbol es

$$\Omega(h) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

donde:

- ▶ T es el número de hojas del árbol;
- ▶ w_j es el valor asignado a la hoja j ;
- ▶ γ penaliza agregar hojas;
- ▶ λ penaliza valores grandes en las hojas.

Interpretación

Un árbol más complejo debe justificar esa complejidad mediante una mejora suficiente en la función objetivo.

Cuidado con la notación

En Gradient Boosting utilizamos η para representar el **learning rate**:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x).$$

En la penalización de XGBoost podemos encontrar λ como parámetro de regularización:

$$\Omega(h) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2.$$

No son lo mismo

η = learning rate, λ = regularización.

Recordemos: Gradient Boosting

Para cada observación calculábamos el gradiente de la pérdida:

$$g_i = \frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i}.$$

Y utilizábamos el gradiente negativo

$$-g_i$$

como dirección para corregir las predicciones.

información de primer orden

El gradiente nos indica en qué dirección debemos modificar la predicción.

XGBoost: información de segundo orden

XGBoost utiliza además la **segunda derivada** de la pérdida:

$$g_i = \frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i},$$

$$h_i = \frac{\partial^2 L(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}.$$

Gradiente g_i

Indica la dirección local en la que cambia la pérdida.

Hessiano h_i

Proporciona información sobre la curvatura local de la pérdida.

Aproximación de la pérdida

Al agregar un nuevo árbol $f_m(x)$, XGBoost aproxima el cambio en la pérdida mediante una expansión de Taylor de segundo orden:

$$L(y_i, \hat{y}_i + f_m(x_i)) \approx L(y_i, \hat{y}_i) + g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2.$$

Por tanto, para construir el siguiente árbol utilizamos información de

primer orden (g_i)

y

segundo orden (h_i).

¿Por qué utilizar segundo orden?

La primera derivada nos indica una **dirección**.

La segunda derivada proporciona información adicional sobre la **curvatura** de la función de pérdida.

gradiente+curvatura $\implies$ mejor información local para optimizar

Esta información se utiliza para:

- ▶ determinar los valores de las hojas;
- ▶ evaluar posibles divisiones;
- ▶ decidir si un nuevo split mejora suficientemente el objetivo.

¿Vale la pena realizar un nuevo split?

Supongamos que estamos considerando dividir un nodo:

$$\boxed{\text{Nodo}} \longrightarrow \boxed{\text{Izquierda}} + \boxed{\text{Derecha}}.$$

XGBoost calcula cuánto mejora la función objetivo al realizar esa división.

$$\boxed{\text{Ganancia} = \text{beneficio de la división} - \text{penalización}}$$

Si la mejora no es suficiente, la división puede rechazarse.

Consecuencia

El árbol no crece simplemente porque sea posible realizar otra división.

Regularización estructural: γ

El parámetro γ controla cuánto debe mejorar la función objetivo para justificar una nueva división.

$\gamma \uparrow \implies$ más difícil agregar nuevas divisiones.

$\gamma \downarrow \implies$ más fácil construir árboles complejos.

Interpretación

γ actúa como una penalización directa sobre el crecimiento del árbol.

Regularización L_1 y L_2

XGBoost permite regularizar los valores asignados a las hojas.

	XGBoost	Tipo
reg_alpha	α	L_1
reg_lambda	λ	L_2

La filosofía es la misma que ya conocemos:

mejor ajuste vs. menor complejidad
--

Valores mayores de regularización restringen la contribución de las hojas.

Muestreo de observaciones

XGBoost también permite que cada árbol utilice solamente una fracción de las observaciones:

subsample

Por ejemplo,

$$\text{subsample} = 0.8$$

indica utilizar aproximadamente el 80% de las observaciones para construir cada árbol.

Esto introduce aleatoriedad y puede funcionar como una forma adicional de regularización.

Muestreo de características

También podemos utilizar solamente una fracción de los predictores:

```
colsample_bytree
```

Por ejemplo,

```
colsample_bytree = 0.8
```

permite utilizar aproximadamente el 80% de las características al construir cada árbol.

Idea

La idea de introducir aleatoriedad sobre las características recuerda a Random Forest, aunque el procedimiento de construcción de los árboles sigue siendo boosting y, por tanto, **secuencial**.

¿Qué tiene de “Extreme”?

XGBoost no sólo modifica la formulación matemática.

También fue diseñado pensando en la **eficiencia computacional**.

Entre sus características se encuentran:

- ▶ algoritmos eficientes para encontrar splits;
- ▶ procesamiento paralelo de operaciones dentro de la construcción de los árboles;
- ▶ manejo eficiente de datos dispersos;
- ▶ optimizaciones de memoria y acceso a datos;
- ▶ soporte para conjuntos de datos grandes.

Importante

Boosting sigue siendo secuencial a nivel de árboles: el árbol m depende del ensamble construido hasta $m - 1$.

Secuencial no significa “sin paralelización”

La dependencia

$$F_{m-1} \longrightarrow F_m$$

impide construir todos los árboles de manera independiente como en bagging.

Pero durante la construcción de **un árbol** existen operaciones que pueden optimizarse y paralelizarse.

árboles secuenciales

≠

todo el cálculo es secuencial

Esta distinción es una parte importante de la eficiencia de las implementaciones modernas.

Otra característica práctica de XGBoost es su capacidad para manejar valores faltantes durante la construcción de los árboles.

Al evaluar una división, el algoritmo puede aprender una **dirección por defecto** para las observaciones con valores faltantes.

$$x_j \text{ faltante} \longrightarrow \text{dirección aprendida}$$

Esto puede simplificar algunos flujos de trabajo, aunque sigue siendo importante comprender **por qué faltan los datos** antes de modelarlos.

Hiperparámetros principales

Podemos agrupar los hiperparámetros según la parte del modelo que controlan.

Aspecto	Hiperparámetros
Número y velocidad	n_estimators, learning_rate
Complejidad del árbol	max_depth, gamma
Muestreo	subsample, colsample_bytree
Regularización	reg_alpha, reg_lambda

Los hiperparámetros interactúan

Por ejemplo, podemos modificar:

```
learning_rate,  n_estimators,  max_depth,  
  
subsample,  colsample_bytree,  
  
gamma,  reg_alpha,  reg_lambda.
```

Además, estos hiperparámetros **no actúan de manera independiente**.

Problema

¿Cómo elegimos una configuración sin utilizar el conjunto de prueba una y otra vez?

XGBoost en Python

Utilizaremos la interfaz compatible con Scikit-Learn:

```
from xgboost import XGBRegressor
```

```
modelo = XGBRegressor(  
    n_estimators=300,  
    learning_rate=0.05,  
    max_depth=3,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    reg_alpha=0.1,  
    reg_lambda=1.0,  
    objective="reg:squarederror",  
    random_state=42  
)
```

```
modelo.fit(X_train, y_train)
```

Lo familiar y lo nuevo

Ya los conocemos

- ▶ `n_estimators`
- ▶ `learning_rate`
- ▶ `max_depth`
- ▶ `subsample`

XGBoost añade

- ▶ `colsample_bytree`
- ▶ `gamma`
- ▶ `reg_alpha`
- ▶ `reg_lambda`

No es necesario ajustar todo a la vez

Una búsqueda de hiperparámetros debe tener un espacio razonable y un presupuesto computacional definido.

Tres formas de trabajar con muchos árboles

	Bagging	Random Forest	Gradient Boosting
Árboles	Independientes	Independientes	Secuenciales
Bootstrap	Sí	Sí	No necesariamente
Variables	Todas	Subconjunto aleatorio	Depende del método
Combinación	Promedio/voto	Promedio/voto	Suma de correcciones
Objetivo principal	Reducir varianza	Reducir correlación	Reducir pérdida

XGBoost pertenece a la tercera familia.

Gradient Boosting clásico vs. XGBoost

	Gradient Boosting	XGBoost
Construcción	Secuencial	Secuencial
Información de pérdida	Gradiente	Gradiente + curvatura
Regularización	Principalmente estructural	Explícita y flexible
Muestreo	Observaciones	Observaciones y variables
Implementación	Clásica	Altamente optimizada

Idea central

XGBoost no abandona Gradient Boosting: **extiende y optimiza su formulación.**

1. Boosting construye modelos **secuencialmente**.
2. Gradient Boosting interpreta cada nuevo árbol como una corrección relacionada con el **gradiente negativo de la pérdida**.
3. El learning rate controla cuánto contribuye cada nuevo árbol:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x).$$

4. XGBoost incorpora información de **segundo orden**, regularización explícita y optimizaciones computacionales.
5. Su flexibilidad introduce muchos **hiperparámetros que interactúan entre sí**.

¿Cómo elegimos esos hiperparámetros?